IN4MATX 285:
Interactive Technology Studio

Programming: Language Roles

Today's goals

By the end of today, you should be able to…

- Differentiate programming languages based on factors such as types and level of abstraction
- Identify the sorts of tasks that a programming language is well-designed for, given a brief description

There are a lot of programming languages out there. They all fill a particular niche.

So many programming languages!

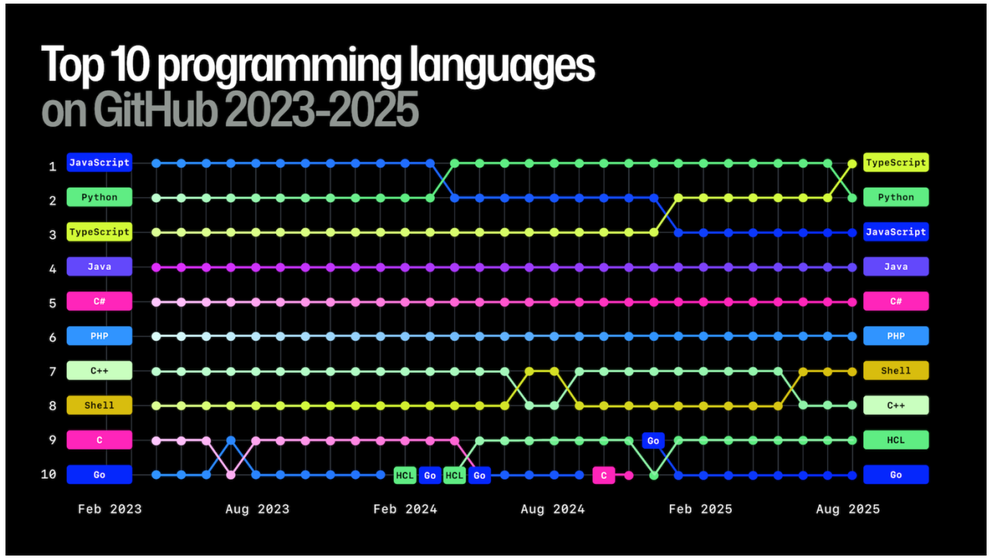
Worldwide, Apr 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	36.21 %	+5.7 %
2	↑↑	C/C++	13.21 %	+6.2 %
3	↓	Java	10.01 %	-5.4 %
4	↑↑	R	6.17 %	+1.6 %
5	↓↓	JavaScript	5.07 %	-3.0 %
6	↑↑↑↑↑	Swift	3.15 %	+0.8 %
7	↓↓	C#	3.0 %	-3.0 %
8		Rust	2.98 %	-0.1 %
9	↓↓	PHP	2.96 %	-0.7 %
10		Objective-C	2.57 %	+0.1 %
11	↑↑↑↑	Ada	2.51 %	+1.1 %
12	↓↓↓	TypeScript	1.8 %	-0.9 %

https://pypl.github.io/PYPL.html

Apr 2026	Apr 2025	Change	Programming Language	Ratings	Change
1	1		Python	20.97%	-2.11%
2	3	↑	C	12.34%	+2.39%
3	2	↓	C++	8.03%	-2.30%
4	4		Java	7.79%	-1.84%
5	5		C#	5.98%	+1.59%
6	6		JavaScript	3.11%	-0.60%
7	8	↑	Visual Basic	3.02%	+0.08%
8	10	↑	SQL	1.75%	-0.44%
9	14	⇈	R	1.62%	+0.43%
10	9	↓	Delphi/Object Pascal	1.52%	-1.01%
11	12	↑	Scratch	1.48%	+0.13%
12	19	⇈	Perl	1.48%	+0.57%

https://www.tiobe.com/tiobe-index/



https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/#the-top-programming-languages-of-2025-typescript-jumps-to-1-while-python-takes-2

So many programming languages!

- Lots of differentiating factors
  - Level of abstraction (high, low)
  - Domain specialization (general-purpose, domain-specific)
  - Runtime and execution model (compiled, interpreted)
  - Programming paradigm (object-oriented, functional)
  - Typing (strong, weak)
- I won’t go through all of these, but you might hear these terms

So many programming languages!

- Today, I'll introduce some of the key differentiators
- I'll spend a bit more time with C/C++ and Python to explain some of the core differences

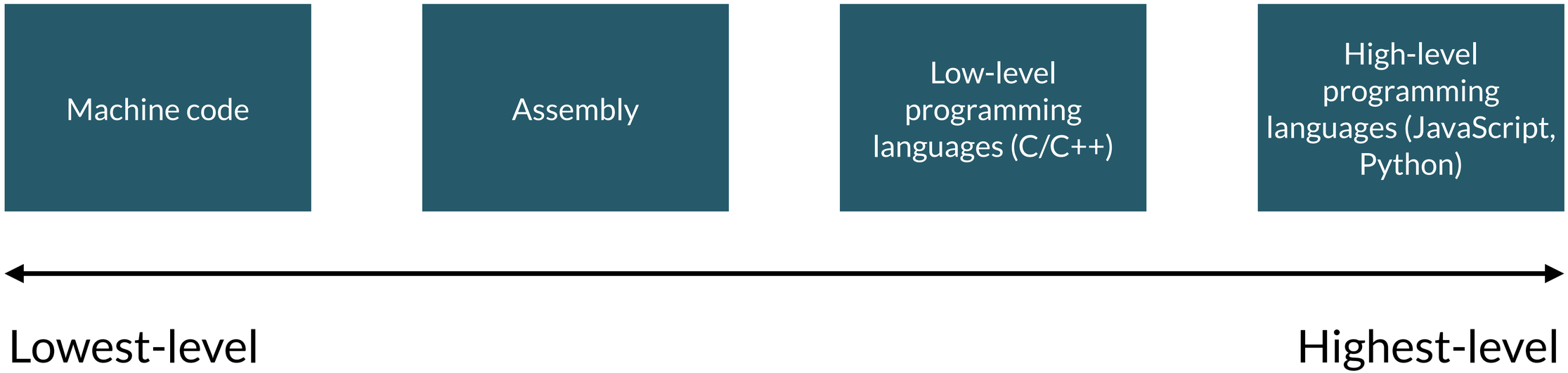
Level of abstraction

Level of abstraction

- Your computer is working entirely in binary (often shown as hexadecimal)
- Any code that you write eventually gets converted to binary
- Some programming languages are *low-level*, or very close to binary, whereas others are *high-level*

Denary	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Level of abstraction



Level of abstraction

Low-level languages

- No or minimal abstraction from the binary code running on a computer
- *Very* performant. Will run fast, won't take up much memory
- All very bad choices if you want to make an interface, but good for other things :-)

https://en.wikipedia.org/wiki/Low-level_programming_language

Level of abstraction

Machine code

- The binary code which can be directly executed on a computer
- Very, very, very few people are programming in machine code
  - Occasionally people will learn to read it to understand memory dumps, like program crashes

```
89 f8
85 ff
74 26
83 ff 02
76 1c
89 f9
ba 01 00 00 00
be 01 00 00 00
8d 04 16
83 f9 02
74 0d
89 d6
ff c9
89 c2
eb f0
b8 01 00 00
c3
```

Fibonacci sequence (1, 1, 2, 3, 5, 8, 13, 21…)

https://en.wikipedia.org/wiki/Low-level_programming_language

Level of abstraction

Assembly

- Human-readable machine code
  - A lot of direct memory manipulation: moving (`mov`) bits around, comparing (`comp`), jumping to specific lines of code (`je`, `jbe`)
- People very occasionally code in assembly if optimization is crucial
  - But, we've gotten very good at optimizing automatically

```
fib:
    mov rax, rdi                ; The argument is stored in rdi, put it into rax
    test rdi, rdi               ; Is the argument zero?
    je .return_from_fib         ; Yes – return 0, which is already in rax
    cmp rdi, 2                  ; No – compare the argument to 2
    jbe .return_1_from_fib      ; If it is less than or equal to 2, return 1
    mov rcx, rdi                ; Otherwise, put it in rcx, for use as a counter
    mov rdx, 1                  ; The first previous number starts out as 1, put it in rdx
    mov rsi, 1                  ; The second previous number also starts out as 1, put it in
rsi
.fib_loop:
    lea rax, [rsi + rdx]        ; Put the sum of the previous two numbers into rax
    cmp rcx, 2                  ; Is the counter 2?
    je .return_from_fib         ; Yes – rax contains the result
    mov rsi, rdx                ; No – make the first previous number the second previous
number
    dec rcx                     ; Decrement the counter
    mov rdx, rax                ; Make the current number the first previous number
    jmp .fib_loop               ; Keep going
.return_1_from_fib:
    mov rax, 1                  ; Set the return value to 1
.return_from_fib:
    ret                         ; Return
```

Fibonacci sequence (1, 1, 2, 3, 5, 8, 13, 21…)

https://en.wikipedia.org/wiki/Low-level_programming_language

C/C++: a low(er) level programming language

C/C++

- C++ is slightly higher-level than C
  - C++ introduces objects similar to those in JavaScript
- Both are widely used and are fairly similar
  - Lowest-level languages which regularly show up in the top 10 most used languages
- I'll talk about them interchangeably, writing code which should* work in both

https://en.wikipedia.org/wiki/C_(programming_language)

https://en.wikipedia.org/wiki/C%2B%2B

C/C++

- Both are especially used for coding:
  - Operating systems
  - Embedded systems (think Raspberry Pi, your smart washer/dryer)
  - Games
- Why?
  - Direct access to hardware and memory
  - High performance

https://en.wikipedia.org/wiki/C_(programming_language) https://en.wikipedia.org/wiki/C%2B%2B

C/C++

- Looks like a lot like JavaScript in syntax (for, if, else, return)
  - Compared to assembly, abstracts away the bits and some aspects of memory

```
unsigned int fib(unsigned int n) {
    if (!n) {
        return 0;
    }
    else if (n <= 2) {
        return 1;
    }
    else {
        unsigned int f_nminus2, f_nminus1, f_n;
        for (f_nminus2 = f_nminus1 = 1, f_n = 0; ; --n) {
            f_n = f_nminus2 + f_nminus1;
            if (n <= 2) {
                return f_n;
            }
            f_nminus2 = f_nminus1;
        }
    }
}
```

Fibonacci sequence (1, 1, 2, 3, 5, 8, 13, 21…)

https://en.wikipedia.org/wiki/C_(programming_language)

https://en.wikipedia.org/wiki/C%2B%2B

C/C++

- But what's an `int`?
  - It's a *type*. Variables in C/C++ must always stay the same type
  - `unsigned int`s must be positive numbers
  - Compared to JavaScript, where variables can change type
- Why types?
  - Reduces errors where code expects one type but receives another
  - Some languages are typed, others aren't

```
unsigned int fib(unsigned int n) {
    if (!n) {
        return 0;
    }
    else if (n <= 2) {
        return 1;
    }
    else {
        unsigned int f_nminus2, f_nminus1, f_n;
        for (f_nminus2 = f_nminus1 = 1, f_n = 0; ; --n) {
            f_n = f_nminus2 + f_nminus1;
            if (n <= 2) {
                return f_n;
            }
            f_nminus2 = f_nminus1;
        }
    }
}
```

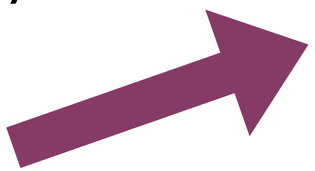
Fibonacci sequence (1, 1, 2, 3, 5, 8, 13, 21…)

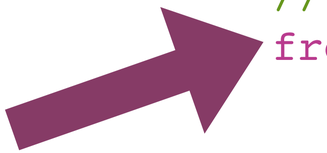
https://en.wikipedia.org/wiki/C_(programming_language)

https://en.wikipedia.org/wiki/C%2B%2B

C/C++

- But C/C++ still requires a lot of memory *management*
- Example: we want to make an array in memory
- Need to directly specify how big it should be
  - E.g., it should hold exactly 5 integers/ numbers
- Once we're done with it, we need to `free` the memory we've allocated





```c
int *arr;

// Allocate memory for 5 integers
arr = (int *)malloc(5 * sizeof(int));

// Initialize and print the array
for (int i = 0; i < 5; i++) {
    arr[i] = i * 10;
    printf("arr[%d] = %d\n", i, arr[i]);
}

// Free the allocated memory
free(arr);
```

https://en.wikipedia.org/wiki/C_(programming_language)

https://en.wikipedia.org/wiki/C%2B%2B

C++

```
➜  06_02_26-language_roles ./a.out
Hello, 0
Hello, 285
➜  06_02_26-language_roles
```

Level of abstraction

Higher-level languages

- If you don’t need direct access to memory, and can sacrifice some performance
- Often viewed as “easier” to code
  - More interpretable
  - More concise

Python: a high(er)-level programming language

Python

- Known for being easy to develop with, having highly readable code, and a lot of rich libraries
- Increasingly becoming the way in which Computer Scientists/Software Engineers are introduced to programming
  - For undergraduates, we teach our first three programming courses in Python
- For me and many others, it’s the language to go to for a relatively-simple task that requires some coding
  - Data processing and organization, basic statistics or visualization

Python

- A few key syntax changes
  - No brackets { }, instead using tabs/ whitespace
  - No semicolons
  - Overall, attempts to be concise

```
def fib(n):
    if n <= 0:
        return 0
    elif n <= 2:
        return 1
    else:
        fib1, fib2, fib_total = 1, 1, 0
        while n > 0:
            fib_total = fib1 + fib2
            if n <= 2:
                return fib_total
            fib2 = fib1
            fib1 = fib_total
            n = n - 1
```

Fibonacci sequence (1, 1, 2, 3, 5, 8, 13, 21…)

Python

- Data structures are highly flexible
- And once you know what you're doing, easy to use to concisely manipulate your data

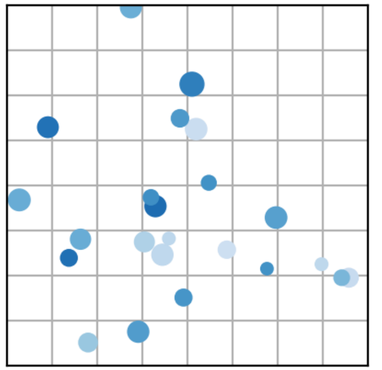
```
titlesAndDois =
[{'title':
paper[‘title'],
'doi': paper[‘addons']
['doi']['url']}
for paper in
UCI_papers if 'addons'
in paper and 'doi' in
paper['addons']]
```

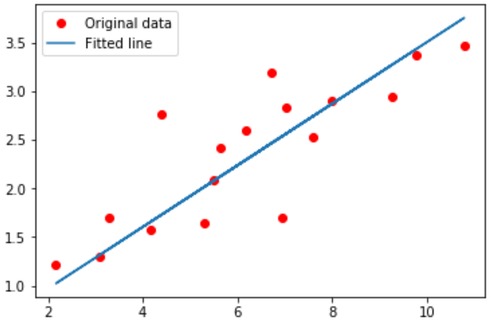
Filters and reshapes a list of objects
Will demonstrate in the demo!

Python

- Really good Data Science and Machine Learning libraries
  - Pandas
  - Numpy
  - Matplotlib
  - Tensorflow

```python
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('_mpl-gallery')

# make the data
np.random.seed(3)
x = 4 + np.random.normal(0, 2, 24)
y = 4 + np.random.normal(0, 2, len(x))
# size and color:
sizes = np.random.uniform(15, 80, len(x))
colors = np.random.uniform(15, 80, len(x))

# plot
fig, ax = plt.subplots()

ax.scatter(x, y, s=sizes, c=colors, vmin=0, vmax=100)

ax.set(xlim=(0, 8), xticks=np.arange(1, 8),
       ylim=(0, 8), yticks=np.arange(1, 8))

plt.show()
```





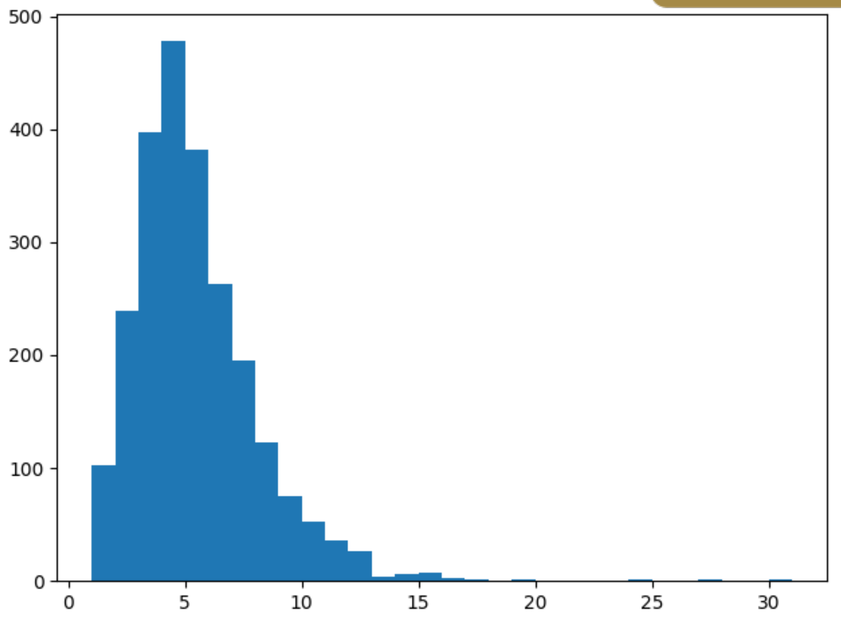

```python
# Weight and Bias, initialized randomly.
W = tf.Variable(rng.randn(), name="weight")
b = tf.Variable(rng.randn(), name="bias")

# Linear regression (Wx + b).
def linear_regression(x):
    return W * x + b

# Mean square error.
def mean_square(y_pred, y_true):
    return tf.reduce_mean(tf.square(y_pred - y_true))

# Stochastic Gradient Descent Optimizer.
optimizer = tf.optimizers.SGD(learning_rate)
```

https://matplotlib.org/

https://www.tensorflow.org/

Python

id	title	doi
188398	‚Äúlt‚Äôs a sp	https://dl.acm.org/doi/10.1145/3706598.3713250
188464	Understandir	https://dl.acm.org/doi/10.1145/3706598.3713197
188591	Understandir	https://dl.acm.org/doi/10.1145/3706598.3713244
188593	A Systematic	https://dl.acm.org/doi/10.1145/3706598.3714072
188660	Understandir	https://dl.acm.org/doi/10.1145/3706598.3713143
188766	What‚Äôs In `	https://dl.acm.org/doi/10.1145/3706598.3713585
188783	Briteller: Shi	https://dl.acm.org/doi/10.1145/3706598.3714106
188907	Understandir	https://dl.acm.org/doi/10.1145/3706598.3713593
188931	Organize, The	https://dl.acm.org/doi/10.1145/3706598.3714193
189011	Meditating To	https://dl.acm.org/doi/10.1145/3706598.3713172
189143	Social by Nat	https://dl.acm.org/doi/10.1145/3706598.3715019
189160	Envisioning t	https://dl.acm.org/doi/10.1145/3706599.3706714
189181	A House Divi	https://dl.acm.org/doi/10.1145/3706598.3713645
189221	"As an Autisti	https://dl.acm.org/doi/10.1145/3706598.3713420
189403	Centering Bl	https://dl.acm.org/doi/10.1145/3706599.3706669
189423	Preparing an	https://dl.acm.org/doi/10.1145/3706598.3713183



Language Benchmarking

```
[➜  06_02_26-language_roles ./demo4                                                ]
Benchmark 1: python demo4_nqueen.py 8
  Time (mean ± σ):      17.6 ms ±   1.8 ms    [User: 13.1 ms, System: 3.5 ms]
  Range (min … max):    16.2 ms …  32.5 ms    85 runs

  Warning: The first benchmarking run for this command was significantly slower than the rest (32.5
ms). This could be caused by (filesystem) caches that were not filled until after the first run. You
 should consider using the '--warmup' option to fill those caches before the actual benchmark. Alter
natively, use the '--prepare' option to clear the caches before each timing run.

Benchmark 2: ./demo4_nqueen_c 8
  Time (mean ± σ):      17.6 ms ±  53.3 ms    [User: 1.2 ms, System: 1.3 ms]
  Range (min … max):     1.4 ms … 202.5 ms    14 runs

  Warning: Command took less than 5 ms to complete. Note that the results might be inaccurate becaus
e hyperfine can not calibrate the shell startup time much more precise than this limit. You can try
to use the `-N`/`--shell=none` option to disable the shell completely.
  Warning: The first benchmarking run for this command was significantly slower than the rest (202.5
 ms). This could be caused by (filesystem) caches that were not filled until after the first run. Yo
u should consider using the '--warmup' option to fill those caches before the actual benchmark. Alte
rnatively, use the '--prepare' option to clear the caches before each timing run.

Benchmark 3: node demo4_nqueen.js 8
  Time (mean ± σ):      26.1 ms ±  14.3 ms    [User: 18.0 ms, System: 5.0 ms]
  Range (min … max):    22.0 ms … 100.3 ms    29 runs

  Warning: The first benchmarking run for this command was significantly slower than the rest (100.3
 ms). This could be caused by (filesystem) caches that were not filled until after the first run. Yo
u should consider using the '--warmup' option to fill those caches before the actual benchmark. Alte
rnatively, use the '--prepare' option to clear the caches before each timing run.

Summary
  ./demo4_nqueen_c 8 ran
    1.00 ± 3.05 times faster than python demo4_nqueen.py 8
    1.49 ± 4.58 times faster than node demo4_nqueen.js 8
➜  06_02_26-language_roles
```

Other languages and wrapping up

Other languages

- PHP 
  - Geared towards web server development, but largely replaced by other languages (JavaScript via NodeJS, Python)
- Java 
  - Useful for being able to run on any device*, but not as designed for web/interfaces as JavaScript and not as performant as C/C++
- R 
  - Used a lot for statistics and data visualization, but few use cases outside of those

My trajectory

- Java in 2005
- C/C++ in 2009
- PHP in 2010
- Python in 2011
- JavaScript in 2013
- R in 2016

- I mostly picked up programming languages when I was in school
  - I’ve never used Rust, Kotlin, Go. Wrote about 5 lines of Swift for an Apple Watch program
- I still use JavaScript and Python regularly, and R on occasion
- I’ve mostly forgotten how to use the others

Overall reflections

- It’s hardest to learn your first language, and relatively easier to learn subsequent ones
- Lots of knowledge transfers between programming languages
  - Syntax stays pretty similar
  - Constructs like loops, variables, and functions are pretty universal
- But languages are often intended to support different use cases, which changes how you use them
  - E.g., DOM manipulation makes JavaScript well-suited to interface development

Overall reflections

- Many tasks *can* be done in a variety of programming languages
  - You can make an interface in C/C++, like with QT (https://www.qt.io/)
  - You can do many low-level memory operations in JavaScript
- But, your life will be *easier* if you choose a language that's well-suited for a particular task
  - Code will be more succinct, or easier to read/understand
  - What you/your team already knows should factor into that choice

Overall reflections

Programming Language design as UX design

- Like anything else, programming languages are designed, and are designed with particular use cases in mind
- Like in interfaces, it’s often possible to do a variety of tasks, but common/important tasks are intentionally easier
- Maybe a bad analogy? You can tell me.

Overall reflections

Languages as community

- Part of what makes a language useful is its community of developers
  - We're all dependent on libraries, and particularly good libraries make a language more appealing
  - e.g., Python has really good libraries for Machine Learning, so people use it for that
  - JavaScript has been extended via React/Vue/Angular for improved interface development
  - Widely-used libraries are increasingly supporting multiple languages
- Chicken and egg question: do a lot of people use a language because it's good, or is a language good because a lot of people use it?

Today's goals

By the end of today, you should be able to…

- Differentiate programming languages based on factors such as types and level of abstraction
- Identify the sorts of tasks that a programming language is well-designed for, given a brief description

IN4MATX 285:
Interactive Technology Studio

Programming: Language Roles